HeavyHeader

面向 C/C++ 构建效能治理的
编译器驱动成本归因平台

项目选题报告

作者：王子周

日期：2026 年 10 月 17 日

申报赛道：第四届开放原子大赛·2026 开源行业解决方案创新赛 · 赛道一（开源基础软件与解决方案）

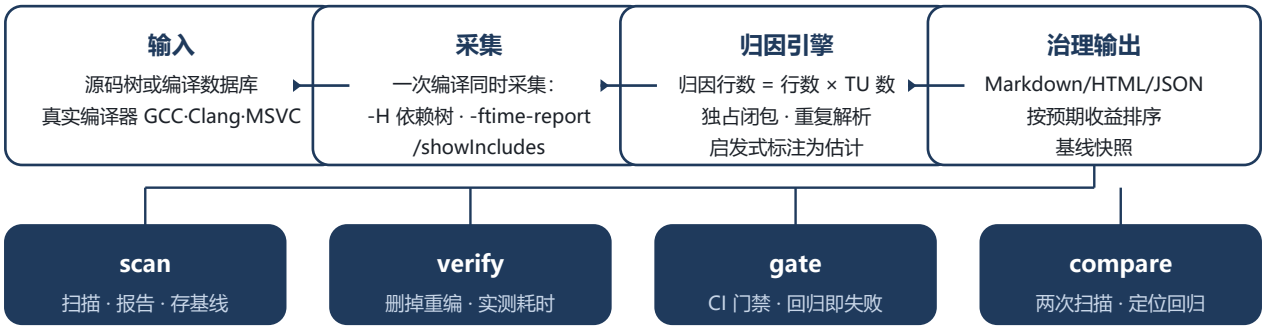


图 1 HeavyHeader 工作链。工具不自己解析 C++，而是向真实编译器提问并读取它的回答，把编译器的真实计时数据与依赖图交叉归因，再交给四个命令消费。图中各环节均已实现并有测试覆盖。

摘要

大型 C/C++ 工程的构建时间是真实的成本：一次全量构建动辄数十分钟到数小时，CI 集群机时与工程师的等待时间都花在里面。而构建慢的最大单一来源，往往不是算法，而是头文件失控——一个被大量编译单元传递包含的重量级头文件，会让整个项目反复为它付费。现有的工具能回答「这个 include 该不该删」，但没有一个工具回答「它到底让我多等了几秒」，因此治理工作缺少排序优先级的依据，也无法在 CI 里防止成本回归。

HeavyHeader 是一个 Python 实现的命令行平台，核心设计只有一条主线：不自己解析 C++。它向真实编译器提问——GCC/Clang 的 `-H` 依赖树与 `-ftime-report` 分阶段计时、MSVC 的 `/showIncludes`——并把编译器的真实输出与依赖图交叉归因。基础指标「归因行数」定义为一个头文件的行数乘以引用它的编译单元数，衡量整个项目为它付出了多少行解析量；在此之上给出独占闭包（删掉某条 include 后真正会停止被解析的集合）、缺少 include guard 的重复解析检测、模板实例化成本与可选的前向声明建议。

四个命令构成完整治理闭环：`scan` 产出 Markdown / HTML / JSON 报告并可存为基线；`verify` 把候选 include 真的注释掉、重新编译、实测耗时差异——成本模型再严谨也仍是模型，权威结论来自实测；`gate` 在 CI 中对基线做回归判定，失败时指名道姓报出是哪些头文件、哪些 include 行引入的；`compare` 对比两份扫描，定位成本上升的具体文件。门禁只使用确定性的归因行数判定，墙钟时间因随负载波动只做展示。

项目当前已完成 v0.1 全部路线图条目、并提前交付 v0.2 的增量扫描、历史看板与 PR 评论机器人，通过 305 个单元测试，CI 覆盖三大操作系统 × 四个 Python 版本。四个真实开源项目（fmt、nlohmann-json、spdlog、cpp-httplib）的基准显示：54 个成功编译单元合计 1,430 万行头文件解析量、87 个成本候选；单体头 httplib.h 一项即被归因42 万行解析量，是最极端的成本热点样本。候选清单只含项目自己的头文件——系统头与第三方头照常计入统计但从不建议删除，因为「建议删掉 <vector>」这类无法执行的条目会把可操作项全部埋掉。所有基准数据可由一条命令复现，失败的编译单元被如实报告而不被跳过。同一份构造工程还已在 GCC / Clang / MSVC 三套真实编译器上跑通横评：三者对项目头的归因行数完全一致（44），差异只落在系统头解析量与耗时上——这正好印证了成本归因与工具链无关，也是「结构指标确定性」的直接证据。

关键词：C/C++；构建效能；头文件依赖；成本归因；编译器计时；持续集成门禁

目录

一 问题背景与需求依据 4
二 平台方案与首版边界 4
三 从扫描到门禁的完整任务 5
四 成本归因如何保持准确 6
五 真实项目基准 7
六 CI 门禁与治理流程 8
七 诚实的边界 8
八 报告与可复现 9
九 技术路线与模块分工 9
十 已有基础与开发起点 10
十一 同类工具与项目价值 11
十二 团队协作与后续开发 12
参考资料 12

一 问题背景与需求依据

构建时间是真实的成本

C/C++ 项目的构建管线里，编译器在每个编译单元上都要重新解析一遍它 include 到的头文件。一个被几万个编译单元传递包含的重量级头文件，会让整个项目反复为同一份文本付费；而这样的头文件往往来自无意的聚合包含、复制粘贴的样板 include，或历史遗留的「保险起见都带上」。构建变慢之后，团队的典型反应是加机器、上缓存（ccache / distcc），这些手段缓解但不消除根因：头文件解析量还在那里，只是被分摊到了更多核与更多命中上。

现有工具回答的是另一个问题

围绕 include 治理已有不少工具，但它们共同缺席了一个问题：成本量化。下表按「能做什么 / 不能做什么」逐项核对。

工具	能做什么	不能做什么
include-what-you-use	基于 AST 判断 include 是否必需	不给成本量化，无法排序优先级
cpp-dependencies	分析模块间依赖关系	面向架构评审，不回答构建耗时
include-gardener	可视化 include 关系图	只展示结构，不给归因
IDE 的「Include Graph」	查看单个文件的包含树	手动、单文件、无项目级视图

缺少成本量化的直接后果是：治理工作没有优先级——几百条 include 里先动哪一条？删错一条省不了一秒，删对一条可能省掉百分之几十；同时治理成果无法沉淀为约束——今天清干净的头文件，明天一个 PR 又把它拖回原样，没有门禁守着。本课题据此把「成本归因 + 实测验证 + 门禁防回归」作为需要同时交付的三件事。R4：include-what-you-use 项目页

面向的用户与代表任务

平台首先面向维护中大型 C/C++ 工程的开发者与构建工程师（Build / Infrastructure 角色），代表任务是：在不破坏构建的前提下，找出让全量构建变慢的头文件，按预期收益排序精简，然后在 CI 中固化成果。输入是源码树（或编译数据库 compile_commands.json）与编译器；输出是按预期收益排序的精简清单、可复现的基准数据，以及一个能在流水线里执行的门禁命令。该任务是首版的需求假设，实际收益通过同任务工具对照与目标用户试用检验。[D1]

二 平台方案与首版边界

一条完整的工作链

工作链从源码树开始：工具发现编译单元（优先采用 compile_commands.json 中的编译命令，否则递归查找源文件并按命令行参数构造），并行调用真实编译器采集依赖树与分阶段计时，在归因引擎里计算每条 include 的成本，最后由四个命令消费同一份数据——扫描报告、实测验证、门禁

判定与两次扫描对比。四路输出共享同一套模型与指标定义，不存在两套数字。[D2]

首版包含的完整范围

首版（v0.1）交付：归因行数、独占闭包、重复解析检测三类确定性归因；模板实例化成本归因（每个编译单元精确、按头文件为启发式）；预编译头收益估算（--pch）与前向声明建议（--forward-decl）两个可选分析；Markdown / 自包含单文件 HTML / JSON 三种报告格式；基线存储与 CI 门禁；以及覆盖四个真实开源项目、一键可复现的基准套件。

编译器	依赖树来源	分阶段计时来源
GCC 9+	-H	-ftime-report
Clang 17+	-H	-ftime-report
MSVC	/showIncludes	墙钟计时（MSVC 无等价的文本格式）

有 compile_commands.json 时严格按其中的 flags 编译——否则同一份代码在不同 flags 下的数字没有可比性。R1：GCC 预处理输出文档 R2：GCC 调试选项文档 R3：MSVC /showIncludes 文档

刻意不做的事

精确判定 include 是否被语义上使用需要 libclang 级别的 AST 分析，那会把使用门槛抬到需要一整套 LLVM 工具链，并破坏「无第三方运行时依赖、Python 3.10+ 即可安装」的部署约束。本课题刻意把精确模式排在后续版本（v0.3）且作为可选依赖，默认路径保持零依赖、能在最朴素的 CI 容器里跑起来。这个取舍与它的理由写在路线图里，不藏在脚注中。[D4]

三 从扫描到门禁的完整任务

扫描与报告

一次典型扫描的终端输出与报告生成：

```
$ heavyheader scan ./src --format html -o report.html
2 TU · 425 条依赖边 · 206,100 行头文件解析量 · 5 个成本候选（其中 1 个疑似未使用）
```

报告按预期收益排序给出成本候选，每个候选标注发现类别与证据——「缺少 include guard」是确定性信号，「疑似未使用」是启发式估计，两者在报告里有明确区分，绝不混排。

实测验证：模型让位于实测

```
## lonely.h <- main.cpp
状态：improved — 基线 520 ms -> 移除后 420 ms
实测节省：100 ms (19.2%)

## core.h <- util.cpp
状态：compile-error — 移除后编译失败：util.cpp:3:14: error: 'Core' was not declared
```

verify 把候选 include 注释掉（不是删除——实验必须可逆）、在临时目录用完全相同的 flags 重新编译、每侧重复 N 次取最小值。编译失败同样是被记录的结果，不会被静默丢弃：它说明这条 include 是必需的，这本身就是有价值的治理结论。

门禁：失败时指名道姓

```
$ heavyheader gate ./src --baseline baseline.json --max-regression 5
头文件解析量：206,100 -> 241,300 (+17.1%)
门禁失败：
- header cost regressed +17.1% (206100 -> 241300 attributed lines), budget is +5.0%
worst offender: include/bloated.h 0 -> 22312 lines (+100.0%)
```

一个只说「成本回归 +12%」的门禁毫无用处；能定位到具体头文件与 include 行的门禁才会被留下来。失败信息直接给出最严重的回归者及其前后行数，构建日志里的每一个人都能立刻知道该找谁、改哪里。

四 成本归因如何保持准确

归因行数：定义与局限

归因行数 = 头文件行数 × 引用它的编译单元数。C++ 项目的预处理与解析成本一阶近似正比于「编译器被迫读了多少头文件文本」，每个被解析的头文件在每个引用它的编译单元里贡献一次自己的行数。这个指标对同一份代码是确定性的，因此适合作为门禁判定依据。它的局限同样明确：行数不等于解析难度，一个 100 行的重型模板头可能比 1000 行的纯声明头更慢——它是相对排序指标，不是绝对耗时的预测器。[D1]

独占闭包：与 lint 工具的本质区别

删掉一条 include 之前必须回答：这个头文件还能不能从别的 include 到达？如果能，删掉当前这条并不会让它停止被解析——钱已经花了。独占闭包定义了删掉某条直接 include 后唯一会停止被解析的头文件集合，只有它才是真实收益。实现上有一个值得记录的真实 bug：早先版本把候选自身也放进了「已付费」并集，导致每个候选的独占集合恒为空，于是所有项目都报告「没有可优化项」——这个症状与「确实没问题」在输出上完全无法区分，因此靠单元测试而非代码审查防守。[D1][D5]

重复解析检测：唯一的「必赚项」

-H 只为编译器实际处理过的包含打印一行，带 guard 的头文件在同一编译单元内物理上不可能出现第二次。因此「同一编译单元内出现多次」必然意味着缺少 include guard，修复（补 #pragma once）永远安全且收益立竿见影——这是全部发现里唯一标注为必赚项的分类。[D1]

模板实例化成本与实测方法学

-ftime-report 从 GCC 9 起报告各阶段耗时，对模板重型 C++，「模板实例化」常是最大单项——实测包含 nlohmann/json.hpp 的文件，39% 的构建时间花在这里。每个编译单元的秒数与占比是精确的；把它按头文件分摊是启发式，报告内明确标注。v

erify 侧的方法学：注释而非删除、临时目录重编并补上原文件所在目录的 -I（#include "..." 相对于包含它的文件所在目录解析）、每侧重复 N 次取最小值——噪声只会让测量变慢不会变快，最小值是最接近真实成本的估计；相对差异低于 2% 视为噪声地板，把 1% 的波动报告成优化成果是不诚实的。R2：GCC 调试选项文档 [D1]

门禁判定公式

```
regression = (attributed_lines_after - attributed_lines_before)
/ attributed_lines_before * 100%
fail if regression > max_regression_percent (默认 5%)
```

默认 5% 刻意保守：在噪声上误触发的门禁会被关掉，而关掉的门禁保护不了任何东西。5% 是「明显变化」与「正常波动」之间的实用分界。[D1]

五 真实项目基准

基准覆盖四个真实开源项目，由 benchmarks/run_all.py 一键生成，每个项目的编译 flags 都是实际跑通、能产出干净数据的那组并记录在案。数据取自 GCC 16.2.0 (MinGW-w64) + Python 3.13，54 个成功编译单元。[D3]

项目	编译单元	依赖边	头文件解析量	成本候选	疑似未使用
fmt	3 成功 / 1 失败	651	331,016	4	0
nlohmann-json	25 成功 / 0 失败	9,294	4,866,676	24	0
spdlog	8 成功 / 17 失败	3,246	1,579,902	41	0
cpp-httplib	18 成功 / 7 失败	14,724	7,523,316	18	0
合计	54 成功	27,915	14,300,910	87	0



图 2 四个真实项目的头文件归因解析量（行）。cpp-httplib 的单体头 httplib.h 一项即被归因 42 万行解析量。数据来源：benchmarks/results/summary.md，可一键复现。

代表性成本热点

项目	头文件	行数	引用 TU	归因行数
nlohmann-json	json.hpp	8,177	24	196,248
cpp-httplib	httplib.h	23,343	18	420,174

项目	头文件	行数	引用 TU	归因行数
spdlog	fmt/bundled/format.h	4,395	8	35,160
fmt	format.h	4,709	2	9,418

读这些数字时要注意三件事。第一，失败的编译单元被如实报告，没有为了凑数字跳过——真实项目常需 CMake 生成的配置头才能编译，本机没有 CMake，spdlog 的 17 个失败单元与 cpp-httplib 缺 OpenSSL 头的 7 个失败单元都写在报告顶部。第二，具体数字随编译器版本、平台、负载变化，不应逐位复现；可复现的是结论方向与数量级。第三，「疑似未使用」是启发式而不是安全性结论，权威判定来自 verify：真的删掉、重新编译。第四，候选清单只含项目自己的头文件；系统头与第三方头照常计入解析量统计——这样各项数字才加得起来——但不进入建议删除的清单。本组数据里「疑似未使用」为 0，正是因为先前计入的那 2 个属于系统头，而判定项目头是否被使用需要 AST 级分析（见第七章与路线图 v0.3）。[D3]

六 CI 门禁与治理流程

从一次性分析到持续治理

单次扫描的价值随时间衰减：报告生成那一刻起，代码就在继续变化。因此平台把「基线快照 + 回归门禁」作为与扫描同等的一等能力。治理流程是：在干净状态扫描并保存基线；此后每个 PR 在 CI 里重新扫描并与基线对比；归因行数回归超过阈值（默认 5%）则流水线失败，失败信息指名道姓。compare 命令支持对任意两份扫描结果做同样的事，用于事后定位「构建什么时候变慢的」。

为什么门禁只用确定性指标

墙钟时间随机器、负载、时钟状态波动，是最吵的数字。gate 不用它做判定依据，只做展示；跨机器、跨时段不比较绝对值。判定依据只有归因行数——它对同一份代码是确定性的，同样的输入必然给出同样的数字，这是门禁可信的前提。趋势图里墙钟时间与归因行数一起画，但归因行数是主指标。[D1]

七 诚实的边界

这个工具不做什么，和它做什么同样重要。以下每一条都是设计决定而非能力缺口，均有对应的理由与替代方案。[D7]

已知局限	含义与对策
不判定 include 是否「未使用」的语义真相	文本启发式会被副作用包含（如 iostream 的流初始化）误判；权威结论来自 verify 实测
前向声明建议必须人工确认	「只以指针/引用形式使用」是语义属性，正则判断可能被宏、typedef、模板参数欺骗；报告给出类型名、行号与阻断类型供人在数秒内确认或否决

已知局限	含义与对策
不做精确的 AST 级分析	精确模式需要 libclang 全套工具链，与零依赖部署约束冲突；排在 v0.3 作为可选依赖
verify 不改写用户代码	实验只在临时目录进行，原始文件一个字节都不会动
不跨机器比较绝对耗时	墙钟时间随负载波动；门禁只对比确定性的归因行数与相对比例
-fsyntax-only 不含代码生成阶段	报告默认不含 opt 与 generate 的成本；--full-compile 可包含但扫描明显变慢
ccache 环境下 verify 需先清缓存	否则测到的是缓存命中时间而不是编译时间

方法论文档为报告里的每一个指标给出定义：「如果一个指标没法在这里查到定义，那它不应该出现在报告里。」[D1]

八 报告与可复现

三种格式，一份数据

scan 同时支持 Markdown（评审与 PR 评论）、HTML（自包含单文件，可直接分享或挂内网）与 JSON（供基线与后续工具消费）三种输出，全部渲染自同一份内部结果，不存在两套数字。HTML 报告内嵌全部样式与数据，无外部资源引用，拷贝到任何机器都能打开。

基准可一键重跑

benchmarks/run_all.py 一键重跑全部四个项目的基准并生成汇总。每个项目的 flags 都记录在基准脚本与结果说明里——写下来是为了下次能复现。compile_commands.json 存在时严格采用其中的编译命令，这是「同一份代码的数字可比」的保障。[D3]

九 技术路线与模块分工

分层结构

模块	职责
drivers/	向编译器提问：GCC / Clang / MSVC 各自的方言
parse/	解析 -H 依赖树、-ftime-report 计时、make 风格依赖文件
attrib/	成本归因：依赖图、独占闭包、重复解析检测、使用启发式
scanner.py	编排：发现编译单元 → 并行编译 → 汇总统计
verify.py	实测验证：注释掉、重编译、计时
baseline.py	基线、对比、CI 门禁
report/	Markdown / HTML（自包含单文件）/ JSON 渲染

几个刻意的设计选择

一次编译同时拿到依赖树、分阶段计时与 guard 缺失三项数据（-H 与 -ftime-report 都写 stderr，可并行收集）——第二次编译会让扫描耗时翻倍，更糟的是两次编译之间头文件若发生变化，两边数据会互相矛盾。扫描默认用 -fsyntax-only 而不是完整编译，换取数量级的速度提升；依赖图上的闭包计算迭代而非递归，喂给畸形输入时不会栈溢出。完整的取舍记录在架构文档里。[D2]

测试策略

305 个单元测试覆盖解析、归因、门禁、verify、增量缓存、多编译器对比与并行编译健壮性各子系统。测试样本优先用真实编译器输出逐字复制而不是手写构造——解析器的输入是编译器方言的文本，手写样本会悄悄偏离真实格式。上一节提到的独占闭包 bug 正是因为「症状与正常输出无法区分」，才以回归测试的方式永久防守。[D5]

真实工程上的完整构建实测

为避免「只有解析量、没有真实构建收益」的质疑，另做了一组可复现的对照实验：对象是 nlohmann/json v3.11.3 的测试目标（63 个编译单元，源码以 SHA-256 固定），两臂各 5 次干净构建。按工具的预编译头建议实施后，完整构建墙上时间中位数 43.2 s → 40.1 s（−7.1%），编译 CPU 292 s → 259 s（−11.4%），63 个测试套件的 4,264,414 条断言逐个一致。[D4]

估算与实测分开陈述、从不合并：工具估算「消掉 91.2% 的头文件解析量」（按行计，是上限），实测是「编译 CPU −11.4%」（按秒计）。由二者反推，头文件解析约占编译 CPU 的 17%——这正是「估算 91% 落地为实测 11%」的原因，也是不能把估算当秒数引用的理由。[D4]

同一实验还得出一个对工具不利的结论，报告里照实写：在「逐个删 include」这件事上，按调用频次排序的经验法则 12 次试验命中 3 个有效改动，而工具按收益排序的 5 个候选全部编译失败。原因是排序只看收益、不看可删性，且词法级用法匹配看不见宏展开。该缺陷与修法已写入路线图 v0.3（改为按「收益 × 可删除概率」排序）。[D4]

十 已有基础与开发起点

当前可以接着做什么

项目不是一个构想：v0.1 路线图全部条目已完成，代码以 Apache-2.0 许可证公开在 github.com/wzzrambler/heavyheader，43 个提交、95 个文件、工作区无未提交改动。下表按「已有基础 / 已核对的内容 / 接下来的开发工作」汇总。[D4]

已有基础	已经核对的内容	接下来的开发工作	
七命令 CLI（scan / verify / gate / compare / history / pr-comment / toolchains）	终端输出样例与 305 个单元测试	PR 评论机器人（pr-comment 子命令，v0.2 已交付，走 GitHub 的 gh）	把两次扫 风格评论
三编译器驱动（GCC / Clang / MSVC）	四真实项目基准，54 个编译单元的采集记录	多编译器对比（toolchains 子命令，v0.2 已交付）	同一份代 阵，支撑
增量扫描（--cache，v0.2 已交付）	文件指纹两级判定；已验证增量结果与全量逐字一致	只重编受改动影响的编译单元 —— 已完成	
三种报告格式与自包含 HTML	HTML 单文件在无外部资源下可打开	历史趋势看板（history 子命令 + --serve，v0.2 已交付）	时间序列 看板，判
CI 门禁与基线	门禁判定公式与 5% 默认阈值的理由记录	按头文件分别设预算：单个头也不许悄悄长胖	
发布流水线	ci.yml 三任务（测试 / lint / self-scan）+ coverage + release	PyPI 正式发版（release.yml 已具备 tag 触发能力）	

持续集成覆盖三大操作系统（ubuntu / windows / macos）× 四个 Python 版本（3.10–3.13），fail-fast 关闭以保证每个组合都被完整验证。self-scan 任务用 HeavyHeader 扫描它自己的源码树——工具吃自己的狗粮。[D5]

十一 同类工具与项目价值

定位差异

与同类工具相比，HeavyHeader 不试图替代它们，而是补上它们共同缺席的一层：成本。IWYU 仍然是最精确的「该不该删」判定者；HeavyHeader 回答的是「删了值不值、先删哪个、删完怎么守住」。

能力	IWYU 等结构分析工具	HeavyHeader
include 必需性判定	精确（AST 级）	启发式 + verify 实测兜底
成本量化与排序	无	归因行数 + 分阶段计时
删掉的预期收益	无	独占闭包精确到头文件集合
实测验证	无	注释-重编-计时，min-of-N
CI 门禁	无	基线 + 阈值回归判定
部署门槛	需 LLVM 工具链	pip 安装，零第三方运行时依赖

项目希望验证的价值主张是：把「头文件成本」从开发者的直觉变成工程数据之后，构建效能治理可以像测试覆盖率一样被持续度量与门禁把守。实际收益通过同任务工具对照（有/无成本数据时的治理效率）与目标用户试用检验，相关方法见方法论文档。[D1]

十二 团队协作与后续开发

申报与协作模式

本项目以开源社区主体类型（个人）申报第四届开放原子大赛·2026 开源行业解决方案创新赛赛道一。项目采用标准的开源协作模式：Issue 报告缺陷与需求、PR 贡献代码、CONTRIBUTING 说明「什么类型的贡献最被需要」、行为规范与安全披露流程齐备。Apache-2.0 许可证对贡献者与使用者都明确，含专利授权条款，且与生态中大量 C++ 项目的许可证兼容。[D6]

后续开发路线

v0.2「从工具到平台」聚焦团队持续使用：本地历史趋势看板、PR 评论机器人（GitHub）、多编译器对比（GCC / Clang / MSVC）均已交付；剩余多人共享的趋势服务（需持久化存储）、GitLab 后端。v0.3「精确模式」引入 libclang AST 后端作为可选依赖，把「疑似未使用」升级为确定结论——默认路径保持零依赖不变。排序即优先级，完整路线图见仓库。[D4]

比赛节点

作品材料（介绍文档 PDF、演示视频、代码仓库）于 2026 年 10 月 31 日 24:00 前提交；11 月 6 日公布入围名单；11 月 13 日总决赛线下路演。当前仓库即评审可见状态，评审期间仅修复缺陷、不引入新功能，保证评委看到的与提交时一致。[D6]

参考资料

外部资料均为原始发布页面或官方文档，访问核对日期为 2026 年 10 月 16—17 日。

· R1 GCC, Preprocessor Output（-H 选项：打印被实际处理的头文件）。https://gcc.gnu.org/onlinedocs/gcc/Preprocessor-Output.html

· R2 GCC, Debugging Options（-ftime-report：编译各阶段计时）。https://gcc.gnu.org/onlinedocs/gcc/Debugging-Options.html

· R3 Microsoft, /showIncludes (List Include Files)。https://learn.microsoft.com/cpp/build/reference/showincludes-list-include-files

· R4 include-what-you-use 项目主页。https://include-what-you-use.org/

· R5 tomtom-international/cpp-dependencies 仓库。https://github.com/tomtom-international/cpp-dependencies

· R6 fedorbondarczyk/include-gardener 仓库。https://github.com/fedorbondarczyk/include-gardener

· R7 nlohmann/json 仓库（基准对象之一）。https://github.com/nlohmann/json

· R8 yhirose/cpp-httplib 仓库（基准对象之一）。https://github.com/yhirose/cpp-httplib

· R9 开放原子开源基金会，开放原子大赛信息页。https://www.openatom.org

项目设计与实测依据

项目内文档按主题列出，全部随公开仓库提交，详细方法、接口与试验记录可供评审查证。

· D1 docs/methodology.md：《方法论：这些数字是怎么算出来的》——全部九个指标的定义与局限。

· D2 docs/architecture.md：《架构》——分层结构、数据流与六个刻意的设计选择。

· D3 benchmarks/：run_all.py 基准脚本与 results/summary.md 汇总——四个真实项目的实测数据。

· D4 ROADMAP.md：《路线图》——v0.1 / v0.2 / v0.3 排序即优先级。

· D5 tests/：305 个单元测试，样本优先采用真实编译器输出逐字复制。

· D6 SUBMISSION.md：《提交清单》——比赛材料、节点与剩余步骤。

· D7 README.md：项目主页——能力总览、诚实的边界与快速开始。

· D8 .github/workflows/：ci.yml（三系统 × 四 Python 版本测试 / lint / self-scan）、coverage.yml（Codecov）、release.yml（tag 触发发布）。